

AI技术应用验证报告

一、验证目的

本报告针对前期软件过程改进方案中提出的“AI 生成测试用例与自动化测试脚本”场景进行原型验证。验证目标是结合实际项目模块，分析 AI 辅助测试设计在测试用例生成效率、测试覆盖范围、自动化程度和缺陷发现能力方面是否具有改进价值。

本次验证的重点，是比较传统人工方式与 AI 辅助方式在测试设计上的差异。传统人工方式主要依据接口文档梳理接口级测试点，关注调用流程、请求参数、返回结构和常见错误；AI 辅助方式则进一步结合项目代码结构，生成 pytest 自动化单元测试，并通过 Mock 大模型、内存仓库和模拟服务隔离外部依赖。通过两种方式的对比，本文将分析 AI 是否能够提高测试用例设计的完整性、自动化程度和问题发现能力，同时说明 AI 生成测试代码后仍需要人工审核和运行验证。

二、验证场景选择

2.1 验证场景

本次选择的验证场景为AI辅助生成测试用例与自动化测试代码，具体对象为多智能体在线语音狼人杀项目中的AI Service模块。

该场景对应前期过程改进方案中的测试验证阶段。按照原有方式，测试人员通常需要先阅读接口文档和相关代码，人工梳理测试点，再继续编写 Mock 对象、测试函数和断言。这个过程需要较多代码阅读和重复性编码工作，尤其是在模块内部逻辑较多、外部依赖较复杂的情况下，测试设计容易停留在接口调用和返回结构检查层面。

引入 AI 技术后，测试人员可以将接口文档、模块代码结构和测试要求提供给 AI 工具，由 AI 辅助生成测试用例、pytest 测试函数、Mock 对象和测试报告。测试人员不再从零开始编写所有测试代码，而是对 AI 生成结果进行审核、运行、修改和补充。这样既保留了人工对测试质量的控制，也能够验证 AI 是否能在测试设计阶段提高效率 and 覆盖范围。

2.2 验证对象与输入材料

本次验证对象为“多智能体在线语音狼人杀系统”中的 AI Service 功能模块。该模块为主游戏后端提供 AI 玩家初始化、角色与人格配置、记忆管理、上下文组装、Prompt 构造、模型结果解析、阶段决策、夜晚技能约束、异常兜底以及多 AI 狼人协同等能力。与普通模块相比，AI Service 内部逻辑更复杂，既涉及游戏规则和角色私有信息，也涉及大模型调用结果的规范化处理，因此更能体现测试设计的难度和 AI 辅助测试生成的价值。

从功能范围看，本次验证主要围绕以下内容展开：

- AI 玩家生成与 LLM 配置绑定；
- AI 角色与人格管理；
- AI 记忆初始化、事件写入和记忆窗口管理；
- 上下文组装与 Prompt 构造；
- 可见事件和私有视野中的敏感字段过滤；
- 模型输出 JSON 解析；
- AI 发言、投票、夜晚技能、猎人开枪等阶段决策规范化；
- 模型调用失败或空输出时的兜底机制；
- 多 AI 狼人夜晚共识与目标聚合。

为了完成本次验证，使用的输入材料包括接口文档、模块代码、测试生成要求和已有运行环境。其中，接口文档用于说明 **AI Service** 的接口范围、请求参数、返回结构、常见错误和主后端接入流程；模块代码用于提供实际被测对象；测试生成要求用于约束 **AI** 输出，使其生成 **pytest** 单元测试；已有运行环境则用于执行测试命令并记录结果。

输入材料	作用
AI Service 接口文档	提供接口列表、请求参数、返回结构、错误情况和主后端接入流程
AI Service 模块代码	提供实际测试对象，包括 <code>service</code> 、 <code>domain model</code> 、 <code>repository</code> 、 <code>prompt helper</code> 等
测试生成要求	要求生成 pytest 单元测试，并使用 Mock 隔离外部模型、数据库和后端服务
已有运行环境	用于执行 pytest 测试命令并记录测试结果

表1 输入材料及作用

2.3 验证工具与测试方式

本次验证采用 **AI** 辅助方式生成测试代码，测试工具为 **codex**，由 **ChatGPT** 作为 **prompt** 设计的辅助工具，测试框架为 **pytest**。由于 **AI Service** 与外部大模型、数据库、房间服务和前后端接口存在交互，如果在单元测试中直接调用这些外部依赖，测试结果容易受到网络、模型输出和运行环境影响。因此，本次验证没有真实调用外部大模型 **API**，也没有连接真实数据库或真实房间服务，而是通过 **Mock**、**stub** 和固定测试数据模拟外部依赖。

在 **AI** 生成的测试代码中，主要使用了 **FakeLLMGateway**、**InMemoryMemoryRepository** 和 **FakeInvokeService** 等测试辅助对象。其中，**FakeLLMGateway** 用于模拟大模型返回固定 **JSON** 或抛出异常，**InMemoryMemoryRepository** 用于在内存中保存和读取 **AI** 记忆状态，**FakeInvokeService** 用于模拟多 **AI** 狼人协同时不同 **AI** 给出的建议结果。

通过这种方式，测试重点可以集中在 **AI Service** 的内部逻辑上，包括 **Prompt** 构造是否正确、敏感信息是否被过滤、模型输出是否能够规范化、非法目标是否会被修正、异常情况下是否能够返回兜底结果等。这样既符合单元测试对隔离性的要求，也能避免真实大模型输出的不确定性影响测试结果。

三、传统人工方式测试设计

为了与 **AI** 辅助方式进行对比，本报告先基于接口文档和模块代码文件梳理传统人工方式下可能形成的基础测试点。人工方式的主要特点是：测试人员先阅读接口文档和模块代码，根据接口路径、请求参数、返回结构和常见错误场景设计测试点，重点关注接口是否能够按预期调用，以及主后端接入流程是否顺畅。

3.1 人工方式测试点清单

编号	测试模块	测试点	预期结果
MT-AI-01	健康检查	调用 GET /health	返回服务状态正常
MT-AI-02	统一响应	调用非 health 接口	返回结构包含 code、message、timestamp、data
MT-AI-03	AI 玩家生成	调用 players/generate 生成指定数量 AI	返回对应数量 AI 玩家，ID 格式为 ai_1、ai_2
MT-AI-04	AI 玩家生成	省略 modelPolicy	使用环境默认模型配置
MT-AI-05	AI ID 规则	使用错误格式 ai1	无法匹配正确 AI 玩家
MT-AI-06	玩家查询	查询某局 AI 玩家	返回该局 AI 玩家列表
MT-AI-07	角色同步	调用 roles/assign	AI 角色被写入
MT-AI-08	角色同步异常	同步不存在 AI 玩家	返回错误或只写入匹配项
MT-AI-09	记忆初始化	调用 memory/init	返回 initialized 列表
MT-AI-10	单个记忆查询	查询指定 AI 记忆	返回 memoryWindow、relations、suspicionScores 等字段
MT-AI-11	记忆查询异常	查询不存在记忆	返回 404 错误
MT-AI-12	公共事件广播	广播 speech 事件	目标 AI 记忆被更新
MT-AI-13	公共事件广播	广播 vote 事件并传 candidateTargets	写入事件并更新可疑度相关信息
MT-AI-14	追加记忆事件	调用 memory/event	返回更新后的 AgentMemoryState
MT-AI-15	人格分配	调用 persona/assign	返回 assignments 和 policies
MT-AI-16	一键初始化	调用 bootstrap	完成玩家生成、记忆初始化和人格分配
MT-AI-17	阶段决策调用	bootstrap 和 roles/assign 后调用 speech	自动补全角色、人格和 LLM 配置
MT-AI-18	决策调用异常	未生成 AI 玩家且省略 llm	返回未绑定 LLM 或未找到 AI 的错误
MT-AI-19	投票阶段	调用 invoke，stage=vote	返回合法投票目标
MT-AI-20	夜晚行动	调用 invoke，stage=night_action	返回 nightAction 或 skillType 相关字段
MT-AI-21	猎人开枪	调用 invoke，stage=death_shot	返回符合猎人技能规则的决策

编号	测试模块	测试点	预期结果
MT-AI-22	异步调用	asyncMode=true 并传 callbackUrl	返回 accepted=true
MT-AI-23	狼人私有视野	传入 wolfTeammates	AI 不应优先攻击或投票给队友
MT-AI-24	狼人共识目标	传入 consensusTarget	夜晚行动优先对齐合法共识目标
MT-AI-25	全 AI 狼人共识	调用 night-consensus	返回 consensusTarget 和 finalKillTarget
MT-AI-26	主后端接入流程	按 bootstrap → roles/assign → broadcast → invoke 顺序调用	主后端能够获得 AI 决策并推进游戏状态
MT-AI-27	私有信息隔离	不通过 broadcast 广播角色私有信息	私有信息应通过 privateVision 传递
MT-AI-28	请求字段错误	传入非法字段类型或枚举值	返回 422
MT-AI-29	LLM 配置安全	查询 players 返回结果	不返回 apiKey
MT-AI-30	STT/TTS 接入	STT 文本广播后再调用 invoke	AI 后续决策能够利用公共发言历史

表2 人工方式测试点清单

3.2 人工方式的局限

从上述清单可以看出，人工方式能够覆盖接口文档中的主要流程和常见异常，适合作为接口联调或黑盒测试设计的基础。但这类测试设计更多依赖于接口文档展开，在进一步形成细粒度单元测试时仍存在一定局限。

- 首先，人工测试粒度偏粗。清单中的内容主要围绕接口是否能调用、参数是否正确、返回结构是否符合预期展开，对模块内部的 service 方法、数据清洗逻辑、Prompt 构造细节和模型输出内容解析等覆盖不足。
- 其次，人工测试点清单本身还不是可执行测试代码。它能够说明“需要测什么”，但还没有完成 Mock 对象、测试数据、断言条件和测试函数的编写。对于 AI Service 这类依赖大模型、记忆仓库和主游戏上下文的模块，要将测试点转化为可运行的单元测试，还需要额外设计 Fake LLM、内存仓库和模拟调用服务，工作量相对较大。
- 此外，复杂边界情况在人工初步设计阶段容易遗漏。例如模型返回空内容、JSON 解析失败、技能重复执行等场景，都需要深入结合代码逻辑和游戏规则才能设计出准确断言。

四、AI辅助方式执行过程

4.1 AI辅助生成思路

本次 AI 辅助方式采用“ChatGPT 辅助设计提示词 + Codex 执行代码生成与测试”的方式完成。由测试人员先准备接口文档、项目代码、测试目标和输出格式要求，再通过与 ChatGPT 交互，设计完整的提示词约束 codex 的工作范围，最后将提示词和相关文件输入 codex，使其围绕 AI Service 模块生成自动化测试代码，并在测试执行后整理测试报告。

具体来说，本次提示词设计主要包含四类约束。第一类是输入材料约束，即要求 AI 先阅读 AI-wolf 目录下与 AI 玩家相关的代码，并结合 AI Service 接口文档理解模块职责。第二类是测试范围约束，即要求测试覆盖 AI 玩家初始化、角色信息管理、上下文构造、Prompt 构造、白天发言生成、狼人夜晚击杀建议、多 AI 狼人协同、模型结果解析、身份暴露过滤和异常兜底等内容。第三类是测试方式约束，即明确要求使用 pytest 编写自动化测试代码，不真实调用外部大模型 API，不连接真实数据库，也不依赖真实房间或前后端服务，所有外部依赖都通

过 Mock、stub 或固定测试数据替代。第四类是输出格式约束，即要求 AI 不仅生成测试代码，还要运行测试并根据执行结果撰写《AI 模块单元测试报告》。

通过上述设计，AI 工具的输出形成了从代码分析、测试设计、测试代码生成、测试执行到报告整理的一组连续活动。测试人员在其中仍然承担过程控制职责，包括准备输入材料、与 ChatGPT 共同完善提示词、限定 Codex 的测试边界、检查生成代码和确认最终测试结果。

4.2 AI辅助生成过程

在实际执行过程中，codex 工具先根据项目目录和接口文档理解 AI Service 模块结构，然后选择 pytest 作为测试框架，为 AI 玩家相关服务生成自动化测试代码。由于本次测试要求聚焦模块内部逻辑，AI 在测试代码中没有直接调用真实大模型、数据库、房间服务或前后端接口，而是使用测试替身模拟外部依赖。

本次 AI 辅助生成过程可以概括为以下步骤：

步骤	内容
第一步	测试人员整理 AI Service 接口文档、AI-Wolf 项目代码位置、测试目标和报告格式要求
第二步	测试人员与 ChatGPT 交互，补充测试范围、Mock 要求、非确定性文本断言方式和报告结构，形成较完整的 Codex 提示词
第三步	将完善后的提示词和相关项目材料提供给 Codex，要求其阅读 AI-Wolf 目录下与 AI 玩家相关的代码
第四步	Codex 根据代码结构识别适合测试的类、函数和 service 方法，包括玩家生成、角色管理、记忆服务、Prompt 构造、模型解析、决策规范化和狼人协同等
第五步	Codex 生成 pytest 测试文件，并构造 InMemoryMemoryRepository、FakeLLMGateway、FakeInvokeService 等测试辅助对象
第六步	Codex 针对正常发言、身份暴露过滤、空上下文兜底、模型调用失败、狼人夜晚目标约束、多 AI 狼人共识等场景生成测试函数
第七步	测试人员检查生成的测试代码，确认其没有真实调用外部大模型、数据库、房间服务或前后端接口，并检查主要断言是否符合测试目标
第八步	运行 <code>python -m pytest tests/test_ai_player.py -q</code> ，记录测试执行结果
第九步	根据测试执行中暴露的问题，对业务代码或测试代码进行修正，并重新运行测试
第十步	测试通过后，Codex 根据测试文件和执行结果整理《AI 模块单元测试报告》，测试人员再对报告内容进行核对和修改

表3 AI辅助生成过程步骤

五、AI辅助生成测试结果

5.1 测试覆盖范围

Codex 根据 AI Service 模块代码生成了自动化测试文件 `tests/test_ai_player.py`。从覆盖范围看，测试不只检查接口能否调用，而是进一步覆盖了 AI 玩家模块中的多个内部服务、工具函数和业务规则。

模块	覆盖内容
PlayerFactoryService	AI 玩家生成、座位分配、模型策略解析、角色写入、配置深拷贝
PersonaService	人格策略范围、人格式样轮询、默认人格 Prompt
MemoryService	记忆初始化、事件转换、去重、窗口裁剪、决策记录、可疑度计算
ContextAssembler	AI 上下文标准字段和别名输出
InvokeService	Prompt 构造、公开上下文提取、私有视野白名单、可见事件敏感字段过滤、发言清洗、阶段决策规范化、异常兜底
LLMGateway JSON 工具	模型文本中 JSON 对象提取、空输出异常
WerewolfTeamService	狼人 ID 去重、合法目标过滤、建议聚合、平局判断、共识写入

表4 AI辅助测试覆盖范围

5.2 测试输出与执行结果

本次 AI 辅助方式最终形成了以下产物：

输出物	内容
自动化测试代码	tests/test_ai_player.py
测试框架	pytest
测试辅助对象	InMemoryMemoryRepository、FakeLLMGateway、FakeInvokeService
测试报告	AI 模块单元测试报告.md
执行命令	python -m pytest tests/test_ai_player.py -q

表5 AI辅助测试输出物与执行命令

测试执行结果如下：

```
python -m pytest tests/test_ai_player.py -q
.....[100%]
47 passed, 29 warnings
```

本次共生成 41 个 pytest 测试函数，其中 1 个角色标准化参数化测试展开为 7 个用例，实际执行合计 47 个自动化单元测试用例。所有测试用例均通过，通过率为 100%。测试过程中出现 29 条 Pydantic V2 迁移相关警告，该警告不影响本轮测试结果，但提示后续可以将模型配置写法迁移到 Pydantic V2 推荐形式。

5.3 AI生成的自动化测试用例

为保证与传统人工方式形成的数据对比，本节列出 AI 辅助方式最终生成并执行通过的自动化测试用例。与人工方式中的测试用例设计清单相比，以下测试不仅包含测试内容和预期结果，还已经落实为可执行的 pytest 测试函数，并通过命令行完成验证。

测试编号	测试内容	输入	预期结果	实际结果	通过与否
UT-AI01	AI 玩家生成、角色写入与 LLM 深拷贝	aiCount=3, modelName="unit-model", 角色分配含 ai_1 与不存在的 ai_9	生成 3 个 AI; 座位递增; 默认人格为 logical; 只写入存在玩家; LLM 返回副本	生成 ai_1/ai_2/ai_3, 座位 [1,2,3]; 只应用 ai_1=werewolf; 修改返回 LLM 副本不影响内部配置	通过
UT-AI02	模型策略 camelCase 字段解析	modelPolicy.modelName、modelPolicy.baseUrl	字段能映射到 model_name 与 base_url	model_name="unit-model", base_url="http://unit.test/v1"	通过
UT-AI03	玩家查询返回深拷贝	生成 ai_1 后修改 get_player() 返回对象	不污染 service 内部玩家对象	外部把副本 role 改为 werewolf 后, 再查内部 role 仍为 None	通过
UT-AI04	人格策略数值范围	PersonaService.DEFAULT_POLICIES	speechRisk、voteVolatility、followGroup 均在 [0,1]	所有人格策略三项数值均在合法范围	通过
UT-AI05	人格轮询分配	["ai_1","ai_2","ai_3","ai_4"]	按 aggressive、conservative、logical 循环	实际为 aggressive、conservative、logical、aggressive	通过
UT-AI06	人格 Prompt 默认值	None 与未知人格字符串	使用 logical Prompt	None 与 unknown-style 返回内容均等于 logical Prompt	通过
UT-AI07	角色标准化: wolf	"wolf"	标准化为 werewolf	返回 werewolf	通过
UT-AI08	角色标准化: werewolf	"werewolf"	标准化为 werewolf	返回 werewolf	通过
UT-AI09	角色标准化: villagers	"villagers"	标准化为 villager	返回 villager	通过
UT-AI10	角色标准化: seer	"seer"	标准化为 seer	返回 seer	通过
UT-AI11	角色标准化: witch	"witch"	标准化为 witch	返回 witch	通过
UT-AI12	角色标准化: hunter	"hunter"	标准化为 hunter	返回 hunter	通过
UT-AI13	角色标准化: 空角色	None	返回 None	返回 None	通过
UT-AI14	记忆初始化	gameId="g1", 玩家 ai_1/ai_2	创建空记忆状态	返回 initialized 为 ["ai_1","ai_2"], 仓库中存在两个状态	通过
UT-AI15	可见事件空内容与重复过滤	空内容事件、重复 speech 事件	空内容跳过, 重复事件只记录一次	memorywindow 仅 1 条, 内容为 same public speech	通过
UT-AI16	记忆窗口裁剪与可疑度计算	max_window=2, 多条发言/投票事件, 候选 p1/p2	只保留最近 2 条; p1 可疑度高于 p2	memorywindow 长度为 2; p1.score > p2.score	通过
UT-AI17	决策记录最多保留 20 条	连续写入 25 条决策	只保留最近 20 条	第一条保留 idx 为 5, 最后一条 idx 为 24	通过
UT-AI18	预言家已查验目标提取	nightAction.inspectTarget=p1 与 skillType=inspect, skillTarget=p2	去重提取查验目标	返回 ["p1","p2"]	通过
UT-AI19	上下文组装字段与别名	AgentProfile、AgentMemoryState、PersonaPolicy、存活玩家	输出标准上下文键与别名字段	包含 self.ai_id、alivePlayers、personaPolicy.speechRisk、currentStage	通过
UT-AI20	角色阶段、公开上下文与敏感字段综合构造	发言请求包含 privateVision.roleMap、visibleEvents.role、secretRole	Prompt 包含角色、阶段、公开上下文; 隐藏身份字段被过滤	user payload 中 role="villager"、stage="speech"、publicContext.selfSeat=3; 敏感字段被移除	通过
UT-AI21	私有视野白名单按角色阶段限制	werewolf、witch night、hunter death_shot、villager	各角色只允许对应有字段	狼人允许 wolfTeammates, 女巫允许 nightDeathCandidate, 猎人允许 hunterCanShoot, 村民为空	通过
UT-AI22	私有视野过滤	狼人请求含 wolfTeammates、allowFriendlyFire、roleMap、allRoles	只保留狼人允许字段	输出仅含 wolfTeammates 与 allowFriendlyFire	通过
UT-AI23	可见事件敏感字段递归过滤	事件含 role、嵌套 team、列表内 secretRole	递归移除隐藏身份字段	输出只保留公开 content、nested.safe、items.text	通过
UT-AI24	公开上下文提取兼容旧 privateVision	privateVision 中传 legacy 上下文字段, 同时请求本身有显式上下文	优先使用请求显式公开上下文	输出 actualStage="day_speech"、selfSeat=3、selfDisplayName="AI Player 1"	通过

测试编号	测试内容	输入	预期结果	实际结果	通过与否
UT-AI25	阶段兜底公开发言	speech、vote、death_shot、night_action	公开阶段兜底非空，夜晚公开文本为空	speech/vote/death_shot 均非空，night_action 返回空字符串	通过
UT-AI26	内部独白与狼人身份泄露检测	“我是AI玩家”、wolf team 文本、正常追问文本	前两者识别为风险文本，正常文本不拦截	AI 元叙事与狼人团队表达返回 True；正常发言返回 False	通过
UT-AI27	Prompt payload 关键字段	狼人发言请求，候选目标 p1/p2	Prompt user payload 包含 stage、role、candidateTargets、publicContext	实际包含 stage="speech"、role="werewolf"、候选 ["p1","p2"] 和发言顺序	通过
UT-AI28	JSON 代码块解析	文本包裹 ``json 代码块   提取 JSON 对象   返回 {"actionType":"speech"}``	通过		
UT-AI29	空模型输出异常	空字符串	抛出 ValueError	实际抛出 ValueError	通过
UT-AI30	普通文本中提取首个 JSON 对象	before {"actionType":"vote","voteTarget":"p1"} after	提取第一个 JSON 对象	返回 {"actionType":"vote","voteTarget":"p1"}	通过
UT-AI31	正常发言输出与后端格式	Fake LLM 返回 speech，并混入 vote/skill/night 字段	发言非空；发言阶段清空非发言字段；输出别名格式正确	actionType="speech"，speechText 非空，voteTarget/skillType/nightAction 均为空，别名输出含 decision.actionType	通过
UT-AI32	狼人白天发言身份暴露过滤	Fake LLM 返回“我是狼人/昨晚刀人/狼队友”表达	公开发言被替换为安全兜底文本	输出非空且不含禁止暴露表达	通过
UT-AI33	空上下文发言补全	空上下文且模型返回空 speechText	返回非空默认发言	speechText 非空，actionType="speech"	通过
UT-AI34	模型失败投票兜底	Fake LLM 抛异常；狼人队友 w1，候选 w1/p1	返回 INVOKE_FAILED；不投队友	fallbackUsed=True，errorCode="INVOKE_FAILED"，voteTarget="p1"	通过
UT-AI35	合法目标排除函数	候选 w1/p1/p2，排除 w1/missing	仅移除存在的排除目标	返回 ["p1","p2"]	通过
UT-AI36	夜晚行动字段归一化	skillType="kill"、skillTarget="p1"	映射到 nightAction.killTarget	killTarget="p1"	通过
UT-AI37	投票阶段非法目标纠正	voteTarget 为 not_alive，候选 p1/p2	自动改为首个候选目标	voteTarget="p1"，技能字段为空	通过
UT-AI38	狼人投票避免队友	狼人 voteTarget 为队友 w1，候选 w1/p1	改投非队友	voteTarget="p1"	通过
UT-AI39	狼人夜晚强制共识目标	模型返回刀队友 w1，共识目标为合法 p2	最终击杀 p2	skillType="kill"，skillTarget="p2"，killTarget="p2"	通过
UT-AI40	预言家避免重复查验	已查验 p1，模型仍查 p1，候选 p1/p2	改查未查验目标 p2	skillType="inspect"，inspectTarget="p2"	通过
UT-AI41	女巫救毒目标限制	刀口 p1，模型救 p2 且毒 p1	救药纠正到刀口；不能毒刀口	saveTarget="p1"，poisonTarget=None，skillType="antidote"	通过
UT-AI42	猎人被毒出局不能开枪	deathReason="poison"，模型开枪 p1	输出 pass，不产生开枪目标	skillType="pass"，skillTarget=None	通过
UT-AI43	缺少 LLM 配置兜底	AgentInvokeRequest.llm=None	不调用模型，返回兜底发言和 INVOKE_FAILED	fallbackUsed=True，errorCode="INVOKE_FAILED"，兜底发言非空	通过
UT-AI44	多狼人协同共识聚合	werewolfAids 含重复值，三名狼人建议 p1/p1/p2	ID 去重，合法目标为非队友；多数建议 p1 成为最终目标；写入记忆	ID 为 ai_1/ai_2/ai_3；legalTargets=["p1","p2"]；finalKillTarget="p1"；三名狼人均记录 night_consensus	通过
UT-AI45	狼人协同工具：ID 去重与队友过滤	ID 列表含重复/空值，候选含队友和 p1	去重后稳定排序；合法目标排除队友	去重为 ["ai_1","ai_2"]；合法目标为 ["p1"]	通过
UT-AI46	狼人基础私有视野重建	privateVision 含旧 consensusTarget/forceConsensusTarget/wolfDecisionMode	移除旧共识字段，写入当前队友	输出 {"x":1,"wolfTeamates":["ai_1"]}	通过
UT-AI47	狼人协同平局检测	两个目标推荐次数与置信度相同	识别为需要候选顺序打破平局	_is_tie_broken_by_order() 返回 True	通过

表6 AI生成的自动化测试用例

5.3 测试中发现并修正的问题

本次 AI 辅助生成测试不仅形成了测试代码，也帮助发现并修正了若干实际问题，说明AI 辅助生成测试不仅提高了测试用例数量，也在一定程度上促进了代码质量改进。

问题	修正
ModelPolicy 未声明 modelName、baseUrl 字段别名，外部传入模型策略时无法稳定绑定指定模型	为 model_name、base_url 增加 alias，并开启 populate_by_name
PlayerFactoryService 对枚举 provider 使用 str()，可能得到 ModelProvider.OPENAI 而不是 openai	增加 provider 归一化逻辑，枚举取 .value
狼人公开发言清洗未覆盖“我是狼人 / 昨晚刀人 / 狼队友”等直白身份暴露表达	在公开发言和投票阶段增加狼人身份暴露正则检测，命中后替换为安全兜底发言

表7 测试中发现并修正的问题

六、传统人工方式与AI辅助方式对比

6.1 对比结果

对比维度	传统人工方式	AI辅助方式
输入材料	接口文档、模块代码	接口文档、模块代码、测试生成提示词
主要工作	人工识别测试对象，整理测试用例设计清单	Codex 根据代码结构生成测试用例、Mock 对象、pytest 测试函数和测试报告
输出形式	30 条测试用例设计清单	tests/test_ai_player.py 自动化测试代码和《AI 模块单元测试报告》
自动化程度	需要后续继续编码才能执行	已形成可运行的 pytest 测试文件
Mock 策略	需要人工后续设计	已生成 FakeLLMGateway、InMemoryMemoryRepository、FakeInvokeService 等测试辅助对象
覆盖层次	覆盖主要功能点和测试方向	进一步覆盖 service 方法、工具函数和内部业务规则
边界场景	能识别部分常见异常场景	覆盖身份暴露、非法目标、重复查验、模型失败、JSON 解析失败、多 AI 狼人协同等细节
用例数量	30 条测试设计项	47 个自动化单元测试用例
执行结果	未形成完整自动化执行结果	47 passed，通过率 100%
问题发现	需要在后续实现和执行中发现问题	测试过程中发现并修正 3 个具体问题

表8 传统人工方式与AI辅助方式对比结果

从数量上看，人工方式形成了 30 条测试设计项，AI 辅助方式最终形成了 47 个可执行自动化单元测试用例。更重要的是，两者的差异不只体现在数量上，还体现在输出形态上：人工方式主要完成“要测什么”的规划，AI 辅助方式则进一步推进到“如何自动化执行测试”。

6.2 对比结果分析

从输出结果看，AI 辅助方式的主要优势在于能够更快地把测试目标转化为可执行测试代码。在传统人工方式下，测试人员虽然能够根据接口文档和代码识别出测试方向，但仍需要继续编写测试数据、Mock 对象、pytest 函数和断言。AI 辅助方式则在提示词约束下，直接生成了测试文件和测试辅助对象，并通过 `pytest` 命令完成执行验证。

在覆盖范围方面，AI 辅助方式不仅覆盖了 AI 玩家生成、角色管理、记忆服务、上下文组装和阶段决策等主要功能，还进一步覆盖了模型输出解析、私有视野过滤、可见事件敏感字段清洗、异常兜底、狼人身份暴露过滤和多 AI 狼人共识等内部逻辑。这说明 AI 在阅读代码上下文后，能够补充一些人工初步设计时不容易一次性展开的细粒度测试场景。

在自动化程度方面，AI 辅助方式的结果更接近可以纳入后续测试流程的过程资产。`tests/test_ai_player.py` 可以通过统一命令运行，后续代码发生修改时也可以作为回归测试使用。相比之下，人工方式形成的测试设计清单仍需要进一步实现，才能进入自动化测试和持续验证流程。

在问题发现方面，本次 AI 辅助测试也产生了实际改进效果。测试过程中发现并修正了模型策略字段别名、`provider` 枚举归一化和狼人公开发言身份暴露过滤等问题。这说明 AI 生成的测试并不只是文档层面的补充，而是能够通过运行结果推动代码质量改进。

6.3 AI辅助方式的局限

AI 辅助方式在本次验证中取得了较好的执行结果，但测试结果仍需要结合测试范围进行理解。47 `passed` 说明已覆盖的自动化单元测试用例全部通过，但并不等同于 AI Service 在所有真实运行场景下都不存在问题。如果测试用例遗漏了关键场景，或者某些断言设计得不够严格，即使测试结果全部通过，也只能说明当前测试范围内没有暴露出错误，不能直接推断模块质量已经完全可靠。

从测试内容看，AI 生成的测试更擅长覆盖代码中能够被识别和拆分的逻辑，例如模型输出解析、私有视野过滤、异常兜底、非法目标修正和狼人协同等内部规则。但狼人杀项目还包含一些较难通过单元测试判断的内容，例如发言策略是否合理、对局平衡是否自然、长上下文推理是否稳定、玩家体验是否符合预期等。这类问题往往需要结合真实大模型调用、完整对局流程、主后端状态推进和前后端联调进行验证。

本次测试采用 `FakeLLMGateway`、内存仓库和模拟服务隔离外部依赖，这有助于提高单元测试的稳定性和可复现性，但同时也意味着测试没有覆盖真实环境中的全部问题。例如真实大模型可能返回格式不稳定的文本，数据库读写可能受到状态持久化影响，主后端状态机推进也可能与 AI Service 的决策结果产生联动问题。因此，本次验证结果更适合说明 AI 辅助方式能够提高单元测试生成效率，而不能替代后续集成测试和系统测试。

此外，AI 生成的测试报告也需要人工复核。AI 可能会根据“全部通过”的结果给出较乐观的总结，因此报告中的结论必须限定在本次测试对象、测试输入和测试执行环境内。对于“AI 模块是否在真实运行中可靠”这一问题，还需要结合后续真实模型调用、复杂对局场景和回归测试结果进一步判断。

综合来看，AI 辅助方式适合作为测试设计和自动化测试代码生成的辅助环节，尤其适合在测试目标明确、代码结构可读取、Mock 边界清楚的情况下使用。但 AI 输出仍需要经过人工审核、运行验证和后续补充测试，才能真正纳入软件过程中的质量控制活动。

七、改进建议

根据本次验证结果，AI 辅助生成测试用例和自动化测试代码具有一定应用价值，但要真正纳入软件过程，还需要配套的输入规范、审核机制和结果记录。后续在测试阶段引入类似方式时，可以从以下几个方面改进。

第一，在使用 AI 工具前，应先准备较完整的输入材料。测试人员需要明确被测模块范围、接口文档、核心业务规则、代码目录位置、测试框架要求以及不允许真实调用的外部依赖。对于 AI Service 这类模块，还应提前说明大模型、数据库、房间服务和前后端接口都需要通过 Mock 或固定数据替代。只有输入材料足够明确，AI 才更容易生成可执行、可复现的测试代码。

第二，提示词设计应作为测试设计活动的一部分进行管理。本次验证中，测试人员先与 ChatGPT 共同完善提示词，再将提示词提交给 Codex 执行，这种方式能够减少 AI 输出范围不清、测试类型混乱或报告格式不一致的问题。后续可以将测试生成提示词沉淀为模板，包含测试范围、测试原则、覆盖场景、断言要求、执行命令和报告格式等内容，作为测试过程资产保留下来。

第三，AI 生成的测试代码必须经过人工审核。测试人员需要检查测试用例是否覆盖核心需求，断言是否有效，Mock 对象是否合理，是否存在重复测试、无效测试或为了通过而弱化断言的情况。尤其是对大模型相关模块，不能只看测试是否通过，还要判断测试是否真正验证了 Prompt 构造、输出解析、敏感信息过滤和异常兜底等关键逻辑。

第四，应区分不同层次的测试目标。本次验证主要覆盖 AI Service 内部逻辑和自动化单元测试，能够验证 service 方法、工具函数和部分业务规则，但不能替代真实大模型集成测试、主后端联调测试、复杂对局测试和用户体验验证。后续测试过程中，可以将 AI 辅助生成的单元测试作为基础，再继续补充接口测试、集成测试和系统测试。

第五，应保留 AI 使用记录和测试修改记录。过程记录至少应包括输入给 AI 的文档和代码范围、提示词内容、AI 生成的测试文件、人工修改内容、测试执行命令、执行结果以及测试中发现的问题。这样既便于后续复盘 AI 辅助测试的实际效果，也能说明最终测试结果经过了人工确认，而不是直接采用 AI 输出。